Generación de Waves

El generador agrupa los productos de los pedidos según su proximidad espacial, teniendo en cuenta el stock disponible, las prioridades y las capacidades máximas de cada wave. Su objetivo es favorecer recorridos de preparación más eficientes.

1. Preparación de registros (PreparacionRegistros.preparar_registros)

Idea

Convertir los ítems de los pedidos en registros producto–pedido con cantidades realmente disponibles. El stock se distribuye primero entre las solicitudes de mayor prioridad.

Estructuras del código

- RegistroProductoPedido: conserva producto, pedido de origen, identificador, prioridad y cantidad asignada.
- Diccionarios: consolidan ítems repetidos de un pedido y controlan el stock disponible.
- Listas ordenadas: establecen un orden determinista para asignar stock y para ingresar a K-Means.

Paso a paso

1. Validar referencias a productos, coincidencia de identificadores y cantidades positivas.
2. Consolidar solicitudes repetidas de un producto dentro del mismo pedido.
3. Ordenar los registros por prioridad descendente y desempatar por identificadores.
4. Asignar stock disponible en ese orden; omitir registros con cantidad cero.
5. Ordenar los registros disponibles por pedido y producto para entregar una entrada reproducible.

Complejidad

Tiempo: O(M log M). Espacio: O(M). Los recorridos y diccionarios tienen costo esperado lineal; dominan los ordenamientos. M es la cantidad de ítems originales.

2. Cálculo de K (CalculoK.calcular_k)

Idea

Estimar la cantidad inicial de agrupaciones espaciales según las capacidades de unidades y de pedidos distintos.

Estructuras del código

- Conjuntos de identificadores de pedidos y de ubicaciones distintas.
- Acumuladores para unidades totales y cantidad de registros.

Paso a paso

1. Contar unidades disponibles, pedidos distintos, registros y ubicaciones.

2. Calcular cuántos grupos serían necesarios por unidades y por pedidos, redondeando hacia arriba.

3. Tomar el máximo de esas necesidades y limitarlo por los registros y ubicaciones disponibles.

$$K = \min(R, U, \max(\lceil Q/C_u \rceil , \lceil P/C_p \rceil))$$

R: registros; U: ubicaciones; Q: unidades; P: pedidos distintos; C_u: límite de unidades; C_p : límite de pedidos.

Complejidad

Tiempo: O(N). Espacio: O(N). Se recorren N registros y se almacenan hasta N identificadores o ubicaciones en conjuntos.

3. Agrupamiento espacial (KMeans.generar_clusters)

Idea

Asignar cada registro al centroide geométricamente más cercano y actualizar los centroides hasta estabilizar las asignaciones o alcanzar el máximo de iteraciones. Se utiliza distancia euclídea cuadrada sobre coordenadas tridimensionales.

Inicialización inspirada en K-Means++

El primer centroide se elige mediante un generador pseudoaleatorio con semilla fija. Los siguientes se seleccionan ponderando las ubicaciones por su frecuencia y su distancia cuadrada al centroide más cercano:

$$peso(p) = frecuencia(p) \times D^2(p)$$

Esto favorece ubicaciones frecuentes y alejadas de los centroides ya elegidos.

Estructuras del código

- Counter: frecuencia de registros por ubicación.
- Lista de centroides y listas de registros por clúster.
- Lista de índices de asignación, utilizada para detectar estabilización.

Paso a paso

1. Obtener ubicaciones y frecuencias; inicializar K centroides mediante selección ponderada.

2. Para cada registro, comparar su distancia a los K centroides y asignarlo al más cercano.

3. Recalcular cada centroide como promedio de las ubicaciones de su clúster.

4. Repetir hasta que las asignaciones no cambien o hasta un máximo de 300 iteraciones; descartar clústers vacíos.

Complejidad

Tiempo: $O(UK^2 + INK)$. Espacio: $O(N)$. La inicialización compara hasta U ubicaciones con centroides seleccionados; cada una de las I iteraciones compara N registros con K centroides. $I \leq 300$.

4. Construcción de waves (ConstruccionWaves.construir_waves)

Idea

Convertir cada clúster en una o varias waves respetando la cantidad máxima de pedidos distintos y unidades. Los registros de clústers diferentes nunca se mezclan en una misma wave.

Estructuras del código

- Lista de registros pendientes y lista de registros reservados para la siguiente pasada.
- Conjunto de pedidos incluidos y contador de unidades acumuladas.
- Objetos Wave, que validan nuevamente las capacidades al incorporar productos.

Paso a paso

1. Ordenar los registros de cada clúster por prioridad descendente y desempates por identificador.

2. Dividir un registro únicamente si su cantidad supera la capacidad máxima de unidades.

3. Recorrer los pendientes e incorporar cada registro cuando no supera las capacidades.

4. Reservar los que no entran para otra wave y repetir hasta agotar los pendientes.

Complejidad

Tiempo: $O(T^2)$ en el peor caso; espacio: $O(T)$. La lista de pendientes puede recorrerse varias veces y Wave.add_product comprueba capacidades recorriendo listas internas. T cuenta registros después de dividir cantidades.

5. Ordenamiento de waves (OrdenamientoWaves.ordenar_waves)

Idea

Priorizar waves con más registros de pedidos de prioridad alta. El ordenamiento no impone que una wave deba terminar antes de iniciar otra.

Estructuras del código

- Perfil de prioridad: tupla de cinco contadores, correspondientes a prioridades 5, 4, 3, 2 y 1.
- Lista de waves ordenada de forma estable.

Paso a paso

1. Contar los registros de cada prioridad dentro de cada wave.
2. Comparar los perfiles lexicográficamente en orden descendente.
3. Conservar el orden original cuando los perfiles son iguales.

Complejidad

Tiempo: O(T + W log W); espacio auxiliar: O(W). Se cuentan los T registros y se ordenan W waves con perfiles de tamaño constante.

6. Cómo se utiliza en el sistema

GeneradorWaves.generar_waves actúa como orquestador: prepara registros, calcula K, genera clústers, construye waves y las ordena. La salida es una lista de waves con productos, cantidades y pedidos asociados.

7. Complejidad general

Variables: M = ítems originales; N = registros disponibles; U = ubicaciones distintas; K = clústers; I = iteraciones; T = registros tras divisiones; W = waves.

Etapa	Tiempo	Espacio
Preparación	O(M log M)	O(M)
Cálculo de K	O(N)	O(N)
K-Means	$O(UK^2 + INK)$	O(N)
Construcción	$O(T^2)$	O(T)
Ordenamiento	O(T + W log W)	O(W)

Tiempo total: $O(M \log M + UK^2 + INK + T^2)$.

Espacio total: O(M + T).

La generación tiene costo polinómico; no prueba todas las combinaciones de productos y pedidos. Los costos con diccionarios y conjuntos se entienden como esperados.

9. Conclusiones

El generador utiliza una estrategia polinómica y modular: prepara los registros disponibles, estima K, realiza un agrupamiento espacial mediante K-Means, construye waves válidas y las ordena por prioridades. La distancia euclídea utilizada para agrupar es una aproximación geométrica y no garantiza recorridos mínimos dentro del almacén.